



Name: Chariesse M. Lobarbio

Role: Leader

- **Specific Tasks performed**

I was responsible for developing the backend, along with one of my group members. What we specifically did in this part was divide the task of creating the endpoints or the PHP files. For my part, I created the logic for the login, register, update_inventory, and delete_from_inventory files. I made sure that the way it was coded would be understandable across different environments or clients to ensure that the REST API would really work.

Aside from that, I also did the testing of the system, making sure that the server and client sides worked together. I checked the two (2) different codes for the client side, which were created by other members of my group, to verify if what they did really consumed the API. When I encountered a problem during testing, what I did was to check everything and make changes once I identified the cause of the problem.

- **Code Contribution**

I created the code and logic for the endpoints that were assigned to me. For all of the PHP files, I made sure they would only accept their allowed methods. For example, in the login and register files, I ensured that they only accept the POST method. If not, they output a status of "error" with a corresponding response code, specifically 405, along with the message "Only POST method allowed." The same logic applies to the other files depending on their required method.

In the login file, I implemented the logic that handles login validation and outputs different responses depending on the result. For example, if someone attempts to login without input data, it outputs a message stating that the username and password are required. If the input data matches the values in the database, it outputs "Login Successfully." It also handles cases where the password is incorrect and checks if the username exists in the database.

For the register file, the logic I created code includes validating whether all required fields (email, pass, first_name, last_name, contact) are filled in. If not, the system returns an error message stating "All fields are required." It also checks if the username is already taken and whether the password meets the minimum requirement of 8 characters. If not, it outputs the corresponding message, requiring users to comply before the system allows account registration.

Lastly, for the update_inventory and delete_from_inventory files, I ensured that when the user (admin) attempts to update or delete a specific product, the inputted inventory_id exists in the database.

- **Challenges Encountered**

There were several problems that I encountered while doing this activity. First, during backend development, I had to ensure that it would really connect to the frontend so that my groupmates handling the frontend would not have difficulty consuming the API. Another major challenge arose during the testing of the overall system functionality. There were times when running the client-side code created by my groupmates resulted in server errors. These issues required me to manually check the logic and identify problems to ensure that the system would output correct responses.

- **Contribution in solving the problem**

I made sure that both client applications worked by checking everything, from paths, to code logic, to responses, ensuring that during the presentation/demo, the system would not fail. Overall, I believe I greatly contributed to completing the task.